

Thirsty's Projects LLC

Sovereign — Source Submission

Word-for-Word Combined User Submission

```
Master Design Plan: Sovereign – Local Coding Agent
Vision
A fully autonomous, locally-operated AI assistant and coding agent. Primary backend

Keeps persistent project context and cross-session memory. Self-improves by synthes

This is designed as a production-grade architecture – not a toy wrapper.

Installed Assets
Asset      Location          Notes
LM Studio   C:\Users\Quencher\.lmstudio\  OpenAI-compatible server at port 1234
Codestral-22B-v0.1-Q2_K      LM Studio      Primary – Mistral code specialist, nat
Gemma-4-E4B-it-Q4_K_M LM Studio      Multimodal fallback, general tasks
Llama-3.2-1B-Instruct-Q8_0    LM Studio      Ultra-fast lightweight tasks
nomic-embed-text-v1.5 T:\models\gpt4all\  Embedding model – semantic memory sea
Python 3.10.11 System PATH      Runtime
Node.js v25.9.0      System PATH      JS execution
Git 2.54.0      System PATH      Version control
T:\ Drive (ReFS)      250 GB, 171 GB free  Primary working drive
Project-AI-main      T:\Project-AI-main\  Source of extracted modules

Core Architectural Pillars
Pillar 1 – Cognitive Engine (Agentic Loop)
The agent runs a ReAct + native function calling loop:

User Input
↓
Intent Classification (extracted IntentDetector from Project-AI)
↓
Model Router (selects Codestral / Gemma / Llama by task type)
↓
Context Assembly (history + injected memories + system prompt)
↓
LLM Call (via LM Studio OpenAI-compatible API)
↓
[Tool Call?] —YES→ Security Gate → Dispatch → Execute → Inject Result → Loop
|
NO
↓
[Final Response] → Render + Append to Audit Log + Save Turn
Loop safety:

Hard cap: 30 tool calls per turn
Timeout: configurable per-tool (default 30s shell, 10s file, 60s web)
Automatic context summarization at 75% of model context window
```

Event Spine routes all cross-component signals (extracted from Project-AI)

Pillar 2 – Multi-Tier Memory System

Four distinct memory layers, all persistent:

Layer	Purpose	Storage	Retrieval
Working	Current conversation context	In-memory list	Window slice
Episodic	Past sessions + outcomes	data/sessions/*.jsonl	By session ID or
Semantic	Facts, preferences, user profile	data/memory/semantic/*.json	By session ID or
Procedural	Learned workflows, how-to patterns	data/memory/procedural/*.json	By session ID or
Vector Search:	nommic-embed-text-v1.5 via LM Studio	/v1/embeddings. FAISS index at d	

At session start, agent automatically injects:

Top 5 semantically relevant past memories for the current topic

User profile summary (preferences, goals)

Most recent episode summary (what happened last session)

Memory writes authorized via a simplified Memory Operational Extensions contract (e

Pillar 3 – Full Tool Ecosystem (45+ Tools)

Filesystem (6 tools)

read_file – read any file; auto-detects encoding

write_file – write file; creates parent directories

list_directory – recursive listing with metadata (size, dates, type)

search_files – glob + content grep across directory trees

move_file – rename or relocate files

delete_file – delete with confirm=true safety flag

Shell & Execution (7 tools)

run_powershell – PowerShell command with stdout/stderr/exit code

run_bash – Bash command (via Git Bash on Windows)

run_python – execute a Python snippet inline, capture output

run_node – execute JavaScript snippet inline

run_script – execute a saved script file

start_background_process – launch a process non-blocking, return PID

kill_process – terminate by PID or name

Code Intelligence (5 tools)

search_code – ripgrep-style search with file type filters + context lines

find_definition – locate function/class/variable definitions across a project

git_status – show working tree status for a repo

git_commit – stage files and commit with message

git_log – show recent commit history with diffs

Thirsty-Lang UTF Toolchain (8 tools)

run_thirsty – execute a .thirsty source file via the extracted interpreter

parse_thirsty – lex + parse + type-check Thirsty-Lang, return AST + errors

eval_tarl_policy – evaluate a TARL policy string against a context dict

validate_shadow_mutation – run Shadow Thirst 6-analyzer suite on a mutation block

encode_tscg – parse and canonicalize a TSCG execution path expression

pack_tscgb – encode a TSCG expression to TSCG-B binary frame

unpack_tscgb – decode and integrity-verify a TSCG-B binary frame

thirsty_format – format/pretty-print Thirsty-Lang source code

Web & Network (4 tools)

web_fetch – HTTP GET/POST, returns clean text (strips HTML boilerplate)

web_search – DuckDuckGo instant answer + organic results (no API key needed)

extract_links – extract all links from a fetched page

download_file – download a URL to a local path with progress

Data & Parsing (4 tools)

parse_json – parse and query JSON with JMESPath expressions

parse_csv – load CSV, return formatted table + stats
 parse_yaml – load YAML files
 convert_format – convert between JSON/CSV/YAML/TOML
 System & OS (5 tools)
 get_system_info – CPU, RAM, disk, network, GPU
 clipboard_read / clipboard_write – system clipboard access
 list_processes – running processes with memory/CPU usage
 notify – Windows toast notification
 get_env – read/list environment variables
 Data Analysis (3 tools, extracted from Project-AI data_analysis.py)
 analyze_csv – compute summary statistics, detect missing values, correlation matrix
 plot_chart – generate matplotlib chart (scatter, histogram, line, heatmap)
 analyze_json – structural analysis + JMESPath querying of complex JSON
 Memory (3 tools, agent-facing)
 save_memory – persist a named memory with tags and content
 recall_memory – retrieve memory by name or semantic query
 list_memories – enumerate all memories with metadata
 Language Database (5 tools)
 query_language_docs – semantic search across the offline language knowledge base (and
 get_syntax_reference – retrieve syntax/grammar docs for a language topic
 list_language_examples – find curated code examples by language + topic
 run_code – execute a code snippet in any supported language (delegates to shell tool)
 get_language_info – overview of a language: paradigm, typing, runtime, use cases
 Introspection & Audit (3 tools)
 show_audit – display recent audit log entries
 replay_session – reconstruct a past session's tool call chain (from Deterministic Repl
 show_metrics – display session metrics (tool calls, tokens, latency)
 Pillar 4 – Thirsty-Lang Runtime Engine
 The agent ships the complete UTF language stack, extracted verbatim from T:\Project-

agent/thirsty/
 |— thirsty_lang/ ← Tier 1: lexer, parser, AST, checker, interpreter, module
 |— thirst_of_gods/ ← Tier 2: OOP extensions (class/async/exception – integrated
 |— tarl/ ← Tier 3: TARL policy parser + evaluator (126 lines, extracted
 |— shadow_thirst/ ← Tier 4: 6-analyzer mutation validator (259 lines, extracted
 |— tscg/ ← Tier 5: symbolic execution graph parser/encoder (150 lines)
 |— tscg_b/ ← Tier 6: binary frame codec + CRC32/SHA-256 integrity (123 lines)
 Syntax Highlighting: agent/ui/terminal.py registers Thirsty-Lang keyword sets in Rich

Language Capabilities the Agent Understands:

Tier What the agent can do
 Thirsty-Lang Write, run, type-check, format, and explain .thirsty programs
 Thirst of Gods Write and explain OOP classes (fountain), async (cascade glass), exceptions
 TARL Write policies, evaluate verdicts against context dicts, explain rules
 Shadow Thirst Write mutations, run all 6 analyzers, interpret PROMOTE/REJECT verdicts
 TSCG Write execution path expressions, canonicalize, checksum, explain each symbol
 TSCG-B Encode/decode binary frames, verify CRC32 + SHA-256 integrity
 Pillar 5 – Offline Language Knowledge Base
 A fully local, semantically-indexed reference database covering 25+ programming languages

Supported Language Roadmap (in learning priority order):

Foundational

Python – general purpose, scripting, data/ML
 JavaScript – web foundation, Node.js runtime

Systems

- C – low-level, memory model, portability
- Rust – memory safety, ownership, systems-level

Enterprise

- Java – OOP, JVM, enterprise patterns
- C# – .NET, Unity, enterprise + game dev

Web Specialization

- TypeScript – typed JS, front-end and Node
- HTML/CSS – markup + styling (treated as a unit)
- Go – cloud-native, back-end, concurrency

Data, AI, Querying

- Python (deep) – NumPy, Pandas, PyTorch, SciKit
- R – statistical computing, data science
- SQL – ANSI standard, joins, indexes, CTEs
- PL/SQL – Oracle procedural SQL
- T-SQL – Microsoft SQL Server dialect

Paradigm Breadth

- Haskell – pure functional, type theory
- Clojure – Lisp on the JVM, immutability
- Scala – hybrid OOP/functional, Spark
- Prolog – logic programming, unification
- Datalog – declarative queries, Datomic

Ops & Automation

- Bash – POSIX shell scripting, pipes
- PowerShell – Windows automation, .NET objects

Mobile

- Swift – iOS/macOS, value types
- Kotlin – Android, JVM, coroutines

Game Dev

- C++ – game engines, unreal, performance

Custom Stack

- Thirsty-Lang – all 6 UTF tiers (separate pillar)

Per-Language Database Structure:

languages/

- └─ {lang}/
 - ├─ overview.md ← paradigm, typing, runtime, ecosystem
 - ├─ syntax.md ← keywords, grammar, operators, control flow
 - ├─ stdlib.md ← standard library index + common APIs
 - ├─ idioms.md ← patterns, best practices, anti-patterns
 - ├─ types.md ← type system, inference, generics
 - ├─ concurrency.md ← threading, async, actors (where applicable)
 - └─ examples/
 - ├─ hello_world.{ext}
 - ├─ data_structures.{ext}
 - ├─ file_io.{ext}
 - └─ error_handling.{ext}

└─ ...

Content Sources (all offline-compatible):

Language	Primary Source Method
Python	pydoc + stdlib python -m pydoc -w batch export
Rust	The Rust Book + rustdoc rustup doc --book (local HTML)
Go	godoc godoc -http :6060 local server export
JS/TS	MDN data npm package npm install mdn-data (offline JSON)
C/C++	cppreference offline Downloaded HTML archive
Java	OpenJDK javadoc javadoc local generation
C#	ECMA-334 spec PDF + markdown extract
Bash	GNU man pages man bash export
PowerShell	Get-Help Get-Help -Full batch export
SQL	ISO SQL standard summary Curated markdown
R	R built-in help help.start() export
Haskell	Haskell 2010 Report PDF + markdown
Swift	Swift.org docs Downloadable DocC
Kotlin	Kotlin docs Offline HTML
All others	Curated markdown refs Hand-authored + verified

Indexing Pipeline (scripts/):

scripts/

- └─ populate_language_db.py ← generates/downloads docs for each language
- └─ index_language_docs.py ← chunks docs, embeds via nomic-embed-text, builds FA
- └─ validate_language_db.py ← verifies all languages have required files

Chunking strategy: 512-token overlapping windows with language + file metadata as FA

Pillar 7 – Project-AI Integration Layer

Modules extracted verbatim or with minimal adaptation from T:\Project-AI-main:

agent/extracted/

- └─ intent_detection.py ← TF-IDF + SGD intent classifier → drives model router
- └─ continuous_learning.py ← fact absorption, idea generation, pros/cons, summar
- └─ audit_log.py ← tamperproof SHA-256 hash-chain audit trail
- └─ event_spine.py ← priority event bus (CRITICAL/HIGH/NORMAL/LOW/DEBUG)
- └─ encrypted_state.py ← AES-256-GCM + ChaCha20 encrypted persistent state
- └─ capability_token.py ← HMAC-SHA256 one-time execution tokens per tool call
- └─ reasoning_trace.py ← records multi-step reasoning chains for explainabil
- └─ deterministic_replay.py ← reconstruct full session from JSONL audit trail
- └─ invariant_check.py ← enforce 4 canonical invariants on tool dispatch
- └─ function_registry.py ← centralized catalog of all available tools + permis
- └─ conversation_context.py ← multi-turn state, topic tracking, reference resolut
- └─ data_analysis.py ← CSV/JSON statistics + matplotlib chart generation
- └─ metrics_collector.py ← tool call counts, token usage, latency, session stat

How these integrate with the agent loop:

intent_detection → classifies each user message as code/general/system/data → select
event_spine → all tool completions fire events that memory + audit log subscribe to
audit_log → every tool call is appended to a hash-chained tamperproof JSONL
capability_token → each tool dispatch requires a freshly minted token (prevents repl
invariant_check → runs before any destructive tool call
reasoning_trace → records the agent's planning steps for multi-turn tasks
continuous_learning → at session end, absorbs new facts the user shared into semanti
deterministic_replay → replay_session tool reconstructs exactly what happened in any

Pillar 8 – Context Management

Context manager tracks token usage (approximate, character-based):

Under 60%: pass full history

60-80%: summarize oldest turns using a single LLM call ("summarize in 200 words")

Over 80%: aggressive compression – keep system prompt + last 3 turns + summary block

Summary stored as an episodic memory for future recall

Conversation Context Engine (extracted from Project-AI) handles topic tracking and recall

Pillar 9 – Task Planner

For complex multi-step requests, the agent enters Plan Mode:

Classify intent (via intent_detection)

Decompose request into ordered subtasks

Present plan to user (with y/n confirmation)

Execute subtasks sequentially with status tracking

On failure, retry once then report the blocking step with reasoning_trace output

On completion, save the workflow as a procedural memory

Task state persisted to data/tasks/active_tasks.json – survives crashes.

Pillar 10 – Model Router

Config-driven model selection by task type:

```
{
  "model_profiles": {
    "code": "lmstudio-community/Codestral-22B-v0.1-GGUF/Codestral-22B-v0.1-Q2_K.gguf",
    "general": "lmstudio-community/gemma-4-E4B-it-GGUF/gemma-4-E4B-it-Q4_K_M.gguf",
    "fast": "hugging-quants/Llama-3.2-1B-Instruct-GGUF/llama-3.2-1b-instruct-q8_0.gguf"
  }
}
```

IntentDetector classifies each prompt and selects the profile automatically. User can override.

Pillar 11 – Plugin System

Tools loaded dynamically from plugins/:

Each plugin implements TOOLS: list[dict] + dispatch(name: str, args: dict) -> dict

Plugins auto-discovered at startup via function_registry – no registration needed

Plugins registered as capability tokens with scoped permissions

Pillar 12 – Security Layer

Path restrictions: allowed_paths in config.json – tools refuse outside dirs by default

Invariant enforcement: 4 canonical invariants checked before any destructive tool call

Capability tokens: every tool dispatch issues a one-time HMAC-SHA256 token (from Project-AI)

Tamperproof audit: SHA-256 hash-chained append-only data/audit/audit.jsonl

Encrypted state: sensitive config values stored via AES-256-GCM (from Project-AI)

Destructive confirmation: delete_file, kill_process, shell with rm/del require confirmation

Pillar 13 – Sub-Agent Delegate System

The agent can spawn independent sub-agent processes (delegates) that run tasks in parallel

Architecture:

Master Agent

```
├── DelegateManager      ← spawns, tracks, terminates delegates
│   ├── DelegateRegistry ← live registry of running delegate PIDs + status
│   └── DelegateRPC      ← JSON-RPC over named pipes (Windows) / Unix sockets
└── Delegates (parallel processes)
    ├── Each delegate is a Python subprocess with its own PTY terminal
    ├── Receives tasks as JSON-RPC calls from master
    └── Has full access to its own tool subset (configurable per template)
```

└─ Sends output, status, and results back to master via RPC
Built-in Delegate Templates (Master Tier Design):

Template	Role	Tools Available
CodeReviewer	Reviews code, flags issues, suggests improvements	read_file, search
TestRunner	Runs test suites, aggregates pass/fail, reports	run_shell, run_p
Refactorer	Applies refactoring patterns across a codebase	read_file, write
SecurityAuditor	Security analysis, vulnerability scanning (Project-AI Red Team)	
DocCompiler	Generates/updates documentation from code	read_file, write_file,
DependencyAuditor	Scans dependencies for CVEs and license issues	run_shell
DataAnalyst	Runs data analysis pipelines, produces charts	parse_csv, analyze_csv,
GitManager	Handles git operations across multiple repos	git_status, git_commit,
ThirstyDev	Specialized Thirsty-Lang development + validation	run_thirsty, par
BuildValidator	Validates builds, runs compilation checks	run_shell, run_script, r

Delegate Pipeline DSL:

Chains of delegates can be defined declaratively:

```
pipeline = Pipeline([
    Delegate("SecurityAuditor", target="T:/Dev/myproject"),
    Delegate("TestRunner", target="T:/Dev/myproject"),
    Delegate("DocCompiler", depends_on=["TestRunner"]),
])
pipeline.run(parallel=True)
New Delegate Tools (6 tools):
```

spawn_delegate – launch a delegate from a named template with a task and target path
list_delegates – show all running delegates + status (running/done/failed)
send_to_delegate – send an additional task or message to a running delegate
get_delegate_output – retrieve stdout/results from a delegate (blocking or polling)
kill_delegate – terminate a delegate by ID
run_pipeline – execute a multi-delegate pipeline definition
IPC Implementation:

Windows: Named Pipes (\\.\pipe\sovereign-delegate-{id})

Fallback: TCP localhost (random port per delegate)

Protocol: newline-delimited JSON-RPC 2.0

Each delegate starts a minimal RPC server and registers with the master on startup

Pillar 14 – Execution Sandbox System

All code execution by the agent (shell, Python, Node, scripts, delegate tasks) can b

Supported Backends:

Backend	How	Security Level	Requirements
local	subprocess + Windows Job Objects	CPU/mem/time limits	Basic Nothing
docker	docker run --rm with hardening flags	High	Docker Desktop
ssh	Execute on remote machine via paramiko	High	SSH target host
singularity	Singularity/Apptainer rootless containers	High	Singularity inst
modal	Modal serverless cloud (optional, breaks offline constraint)		Managed

Docker Hardening Profile (default when Docker backend selected):

```
docker run --rm \
  --network=none \
  --read-only \
  --tmpfs /tmp:rw,size=256m \
  --memory=512m --cpus=1.0 \
```

```
--pids-limit=64 \
--security-opt no-new-privileges \
--security-opt seccomp=agent/sandbox/seccomp_profile.json \
--cap-drop ALL \
-v {workspace}:/workspace:ro \
{image} {command}
```

Windows Local Hardening (Job Objects):

CPU time limit (default 30s per command)

Memory limit (default 512 MB)

Process count limit (64 child processes)

No elevated privilege inheritance

Kill-on-job-close enforced

Sandbox Module Structure:

```
agent/sandbox/
├─ __init__.py          ← SandboxFactory + backend selection
├─ base.py              ← SandboxBackend ABC
├─ local.py             ← LocalSandbox (subprocess + Job Objects on Windows)
├─ docker.py            ← DockerSandbox (docker run with hardening)
├─ ssh.py               ← SSHSandbox (paramiko remote execution)
├─ singularity.py       ← SingularitySandbox (rootless containers)
├─ modal.py             ← ModalSandbox (optional cloud, disabled by default)
└─ seccomp_profile.json ← restricted syscall profile for Docker backend
```

Config-driven backend selection:

```
{
  "sandbox": {
    "default_backend": "local",
    "docker_image": "python:3.12-slim",
    "ssh_host": null,
    "memory_limit_mb": 512,
    "cpu_limit": 1.0,
    "timeout_seconds": 30
  }
}
```

Switch to Docker: change default_backend to "docker". All run_shell, run_python, run.

Pillar 15 – Persistent Project Memory

The agent maintains per-project context that survives across sessions. When the agent

Project Context Structure:

```
data/projects/{sha256_of_path}/
├─ context.json          ← file tree snapshot, git branch, open todos, last modified
├─ conventions.md        ← coding style, naming patterns, linting rules, test framework
├─ history.md            ← rolling summary of work done on this project (auto-updated)
└─ notes.jsonl           ← append-only log of project-specific observations
```

Auto-detection: On every session start, if cwd or any referenced path matches a known

New tools (4):

load_project_context – manually load a project profile by path

save_project_note – append a note to the current project's log

update_project_conventions – update the coding convention doc for a project

list_projects – show all known project profiles with last-accessed dates

Pillar 16 – Self-Improving Skill Engine

The agent observes its own behavior across sessions and synthesizes reusable skills

Skill Lifecycle:

Observation (pattern detected in session logs)

↓

PatternDetector – finds $N \geq 3$ repetitions of a task sequence

↓

SkillSynthesizer – generates Python skill code via LLM

↓

SkillTester – runs skill against a test case, validates output

↓

SkillRegistry – stores approved skill in skills/

↓

SkillRefinement – after each use, delta is logged; every 5 uses, re-synthesize

Skill Storage:

skills/

├─ {skill_name}.py ← skill implementation (callable Python)

├─ {skill_name}.meta.json ← name, description, tags, use_count, last_refined, score

└─ index.json ← searchable catalog of all skills

New tools (4):

run_skill – invoke a named skill with arguments

list_skills – show all skills with metadata (use count, score, tags)

synthesize_skill – manually trigger skill synthesis from a described pattern

improve_skill – trigger a refinement pass on an existing skill

Pattern Detection: PatternDetector scans data/sessions/ JSONL logs for repeated tool

Pillar 17 – Multi-Platform Gateway

An optional gateway service that routes messages from external platforms to the agent

Architecture:

Platform Connectors (async coroutines)

├─ TelegramConnector – python-telegram-bot

├─ DiscordConnector – discord.py

├─ SlackConnector – slack_bolt

├─ WhatsAppConnector – twilio (WhatsApp API)

├─ SignalConnector – semaphore or signal-cli subprocess

└─ EmailConnector – imaplib (receive) + smtplib (send)

↓

GatewayRouter

├─ AuthFilter – allow-list of sender IDs per platform

├─ RateLimiter – max N messages/minute per sender

├─ MessageQueue – asyncio.Queue → agent input

└─ ResponseDispatcher – routes agent response back to originating platform

↓

Agent Core (unchanged)

Gateway is started as a separate process:

```
python main.py --gateway          # start agent + all configured gateways
```

```
python main.py --gateway telegram discord # start specific connectors only
```

Config (per platform, all optional):

```
{
  "gateways": {
    "telegram": { "enabled": false, "token": "...", "allowed_users": [123456] },
    "discord": { "enabled": false, "token": "...", "allowed_guilds": ["..."] },
    "slack": { "enabled": false, "bot_token": "...", "app_token": "..."},
    "whatsapp": { "enabled": false, "account_sid": "...", "auth_token": "..."},
    "signal": { "enabled": false, "phone": "+1...", "allowed_senders": ["+1..."] },
    "email": { "enabled": false, "imap_host": "...", "smtp_host": "...", "user": ... }
  }
}
```

All tokens stored in AES-256-GCM encrypted state. Gateway connectors are optional dependencies.

New tools (2):

send_platform_message – send a message to a user on a connected platform
 list_gateway_status – show which platform connectors are running + message counts

Pillar 18 – Multi-Model Provider Support

The model router supports any OpenAI-compatible endpoint. Provider selection is configurable.

Supported Providers:

Provider	Base URL	Notes
LM Studio	http://localhost:1234/v1	Primary – fully offline
Ollama	http://localhost:11434/v1	Local fallback
OpenRouter	https://openrouter.ai/api/v1	3rd party
Nous Portal	https://inference-api.nousresearch.com/v1	3rd party
NovitaAI	https://api.novita.ai/v3/openai	3rd party
NVIDIA NIM	https://integrate.api.nvidia.com/v1	3rd party
OpenAI	https://api.openai.com/v1	GPT-4o, o1, etc.
MiniMax	https://api.minimax.chat/v1	MiniMax
Custom	User-defined URL	Any OpenAI-compatible endpoint

Provider Config (supports fallback chain):

```
{
  "providers": {
    "primary": { "name": "lmstudio", "base_url": "http://localhost:1234/v1",
    "fallback": { "name": "openrouter", "base_url": "https://openrouter.ai/api/v1",
    "fast": { "name": "novitaai", "base_url": "https://api.novita.ai/v3/openai"
  },
  "model_profiles": {
    "code": { "provider": "primary", "model": "lmstudio-community/Codestral-22B",
    "general": { "provider": "primary", "model": "lmstudio-community/gemma-4-E4B",
    "fast": { "provider": "primary", "model": "hugging-quants/Llama-3.2-1B-Instruct",
    "cloud": { "provider": "fallback", "model": "anthropic/claude-sonnet-4-6" }
  }
}
}
```

Failover logic: If the primary provider returns an error or timeout, the router automatically falls back to the next provider in the chain.

Pillar 19 – One-Command Install & Interactive Setup Wizard

Install flow (single command):

```
# Clone and install
git clone <repo> T:\Dev\Sovereign && cd T:\Dev\Sovereign && .\install.ps1
install.ps1 does exactly this in order:
```

Check Python ≥ 3.10, Git, pip – alert if missing

sovereign/updater.py – sovereign update logic

CHANGELOG.md – version history

Repository Structure

T:\Dev\Sovereign\

```
|— .git/
|— .gitignore
|— README.md
|— CHANGELOG.md
|— pyproject.toml
|— install.ps1           ← Windows one-command installer
|— install.sh            ← Bash/WSL installer
|— config.json           ← backend, model profiles, paths, security
|— system_prompt.md      ← agent identity and behavioral instructions
|— main.py               ← CLI entry point (sovereign command)
|
|— sovereign/            ← installable Python package (matches pyproject.toml)
|   |— __init__.py
|   |— core.py           ← agentic loop, tool dispatch, safety caps
|   |— config.py         ← config loading + env resolution
|   |— context.py        ← context window tracking + summarization
|   |— planner.py        ← task decomposition + subtask executor
|   |— router.py         ← model selection by intent class
|   |— setup_wizard.py   ← interactive setup wizard (sovereign setup)
|   |— updater.py        ← sovereign update logic
|   |
|   |— tools/
|       |— __init__.py   ← registry, OpenAI schemas, dispatch (75+ tools)
|       |— filesystem.py ← file CRUD + directory ops
|       |— shell.py      ← PowerShell, Bash, Python, Node, scripts
|       |— web.py        ← fetch, search, extract, download
|       |— code.py       ← code search, git operations
|       |— system.py     ← processes, clipboard, notify, env vars
|       |— data.py       ← JSON/CSV/YAML/TOML parsing + analysis
|       |— thirsty.py    ← all 8 Thirsty-Lang UTF tools
|       |— language_db.py ← 5 language database tools (query, examples, etc)
|       |— delegates.py  ← 6 delegate management tools
|       |— sandbox.py    ← run_in_sandbox + backend switching
|       |— skills.py     ← 4 skill engine tools
|       |— gateway.py    ← 2 gateway tools
|       |— projects.py   ← 4 project context tools
|       |— introspection.py ← audit, replay, metrics tools
|   |
|   |— memory/
|       |— __init__.py
|       |— store.py      ← unified memory interface
|       |— episodic.py   ← session history + recall
|       |— semantic.py   ← facts, preferences, knowledge
|       |— procedural.py ← learned workflows + task patterns
|       |— vector.py     ← nomic-embed-text, FAISS, semantic search
|   |
|   |— thirsty/          ← Thirsty-Lang UTF runtime (extracted from Project Thirsty)
|       |— __init__.py
|       |— thirsty_lang/ ← T1: lexer, parser, AST, checker, interpreter
|       |— tarl/         ← T3: policy evaluator
|       |— shadow_thirst/ ← T4: mutation validator (6 analyzers)
|       |— tscg/         ← T5: symbolic execution graph
```

└─ tscg_b/	← T6: binary frame codec
└─ extracted/	← Modules extracted from Project-AI-main
└─ └─ __init__.py	
└─ └─ intent_detection.py	← TF-IDF + SGD intent classifier
└─ └─ continuous_learning.py	← fact absorption engine
└─ └─ audit_log.py	← tamperproof hash-chain audit trail
└─ └─ event_spine.py	← priority event bus
└─ └─ encrypted_state.py	← AES-256-GCM persistent state
└─ └─ capability_token.py	← one-time execution tokens
└─ └─ reasoning_trace.py	← multi-step reasoning recorder
└─ └─ deterministic_replay.py	← session reconstruction
└─ └─ invariant_check.py	← mechanical constraint enforcement
└─ └─ function_registry.py	← capability catalog
└─ └─ conversation_context.py	← multi-turn context engine
└─ └─ data_analysis.py	← CSV/JSON analysis + charts
└─ └─ metrics_collector.py	← performance metrics
└─ security.py	← path validator, command filter, security lay
└─ skills/	← self-improving skill engine
└─ └─ __init__.py	
└─ └─ pattern_detector.py	← identifies repeated task sequences in sessi
└─ └─ synthesizer.py	← generates skill code via LLM
└─ └─ tester.py	← validates synthesized skills
└─ └─ registry.py	← skill catalog + index
└─ gateway/	← multi-platform message gateway
└─ └─ __init__.py	
└─ └─ router.py	← GatewayRouter + AuthFilter + RateLimiter
└─ └─ connectors/	
└─ └─ └─ telegram.py	
└─ └─ └─ discord.py	
└─ └─ └─ slack.py	
└─ └─ └─ whatsapp.py	
└─ └─ └─ signal.py	
└─ └─ └─ email.py	
└─ └─ dispatcher.py	← routes responses back to platforms
└─ providers/	← multi-model provider routing
└─ └─ __init__.py	
└─ └─ router.py	← ProviderRouter + fallback chain
└─ └─ lmstudio.py	
└─ └─ ollama.py	
└─ └─ openrouter.py	
└─ └─ nous.py	
└─ └─ novitaai.py	
└─ └─ nvidia_nim.py	
└─ └─ openai.py	
└─ └─ minimax.py	
└─ └─ custom.py	
└─ projects/	← persistent per-project context
└─ └─ __init__.py	
└─ └─ detector.py	← identifies active project from cwd
└─ └─ profile.py	← load/save project profiles

- └─ updater.py ← updates context.json + history.md at session
- └─ delegates/ ← sub-agent delegate system
 - └─ __init__.py
 - └─ manager.py ← DelegateManager: spawn, track, terminate
 - └─ rpc.py ← JSON-RPC 2.0 over named pipes / TCP
 - └─ pipeline.py ← Pipeline DSL for chaining delegates
 - └─ templates/
 - └─ __init__.py
 - └─ code_reviewer.py
 - └─ test_runner.py
 - └─ refactorer.py
 - └─ security_auditor.py
 - └─ doc_compiler.py
 - └─ dependency_auditor.py
 - └─ data_analyst.py
 - └─ git_manager.py
 - └─ thirsty_dev.py
 - └─ build_validator.py
- └─ sandbox/ ← execution sandbox backends
 - └─ __init__.py ← SandboxFactory + backend selection
 - └─ base.py ← SandboxBackend ABC
 - └─ local.py ← LocalSandbox (subprocess + Windows Job Object)
 - └─ docker.py ← DockerSandbox (hardened docker run)
 - └─ ssh.py ← SSHSandbox (paramiko remote execution)
 - └─ singularity.py ← SingularitySandbox (rootless containers)
 - └─ modal.py ← ModalSandbox (optional cloud, off by default)
 - └─ seccomp_profile.json ← restricted syscall profile for Docker
- └─ ui/
 - └─ __init__.py
 - └─ terminal.py ← rich TUI + Thirsty-Lang syntax highlighting
- └─ plugins/ ← user-extensible tool plugins
 - └─ __init__.py
 - └─ example_plugin.py
- └─ languages/ ← offline language knowledge base
 - └─ python/
 - └─ javascript/
 - └─ typescript/
 - └─ c/
 - └─ cpp/
 - └─ rust/
 - └─ java/
 - └─ csharp/
 - └─ go/
 - └─ r/
 - └─ sql/
 - └─ plsql/
 - └─ tsql/
 - └─ haskell/
 - └─ clojure/
 - └─ scala/
 - └─ prolog/

```

├── datalog/
├── bash/
├── powershell/
├── swift/
├── kotlin/
├── html_css/
├── thirsty_lang/           ← cross-ref to agent/thirsty/ docs
├── scripts/                ← database population + indexing
│   ├── populate_language_db.py ← downloads/generates docs per language
│   ├── index_language_docs.py  ← chunks + embeds into FAISS
│   └── validate_language_db.py ← ensures all languages are complete
├── data/
│   ├── memory/
│   │   ├── semantic/         ← {name}.json files
│   │   ├── episodic/         ← session summaries
│   │   └── procedural/       ← workflow patterns
│   ├── sessions/            ← {session_id}.jsonl (full turn logs)
│   ├── vectors/
│   │   ├── memory.index      ← FAISS index for agent memory
│   │   ├── memory_id_map.json
│   │   ├── language_docs.index ← FAISS index for language database
│   │   └── language_id_map.json
│   ├── tasks/               ← active_tasks.json
│   ├── projects/            ← per-project context profiles
│   │   └── {sha256_of_path}/
│   │       ├── context.json
│   │       ├── conventions.md
│   │       ├── history.md
│   │       └── notes.jsonl
│   └── audit/                ← audit.jsonl (tamperproof hash-chain)
├── skills/                  ← synthesized reusable skill scripts
│   ├── {skill_name}.py
│   └── {skill_name}.meta.json
Dependencies (pyproject.toml)
[project]
name = "sovereign"
version = "0.1.0"
description = "Sovereign – Autonomous Local Coding Agent & Personal Assistant"
requires-python = ">=3.10"

[project.scripts]
sovereign = "sovereign.main:app"      # sovereign run / setup / update / gateway

dependencies = [
    "openai>=1.50",          # LM Studio / Ollama OpenAI-compatible client
    "rich>=13",              # Terminal UI: markdown, syntax, panels, progress
    "typer>=0.12",           # CLI argument parsing
    "faiss-cpu>=1.8",        # Vector similarity search for semantic memory
    "numpy>=1.26",           # FAISS dependency
    "scikit-learn>=1.4",     # TF-IDF + SGD for intent detection (from Project-AI)
    "duckduckgo-search>=6",  # Web search, no API key required
    "httpx>=0.27",           # HTTP client
    "beautifulsoup4>=4.12",  # HTML content extraction

```

```

"pyyaml>=6",           # YAML parsing
"toml>=0.10",          # TOML parsing
"pywin32>=306",        # Windows clipboard + notifications
"gitpython>=3.1",      # Git operations
"cryptography>=42",    # AES-256-GCM, Ed25519, ChaCha20 (from Project-AI)
"matplotlib>=3.8",     # Chart generation (from Project-AI data_analysis)
"paramiko>=3.4",       # SSH sandbox backend
"docker>=7.0",         # Docker sandbox backend (requires Docker Desktop)
"jsonrpcclient>=4.0",  # JSON-RPC client for delegate communication
"jsonrpcserver>=5.0",  # JSON-RPC server for delegate processes
]

```

[project.optional-dependencies]

```

gateway-telegram = ["python-telegram-bot>=21"]
gateway-discord  = ["discord.py>=2.4"]
gateway-slack    = ["slack_bolt>=1.19"]
gateway-whatsapp = ["twilio>=9"]
gateway-email    = [] # stdlib only (imaplib + smtplib)
gateway-all      = ["python-telegram-bot>=21", "discord.py>=2.4", "slack_bolt>=1.19"]
CLI Interface (sovereign command)

```

After install, Sovereign is available as the sovereign command:

```

sovereign          → start new interactive session (default)
sovereign run      → same as above (explicit subcommand)
sovereign setup    → run interactive setup wizard
sovereign update   → pull latest code + re-index language DB
sovereign gateway [telegram discord ...] → start platform gateway connectors
sovereign --resume <session_id>          → continue a session
sovereign --list-sessions                  → show all sessions with timestamps
sovereign --model code|general|fast|cloud → force model profile
sovereign --provider lmstudio|openrouter|novitaai|nim|openai|minimax
sovereign --no-memory                      → run without memory injection
sovereign --project <path>                 → load a specific project context

```

In-session slash commands:

```

/help          → show available commands
/memory        → list saved memories
/forget <name> → delete a memory
/model <name>  → switch model profile mid-session
/plan         → show current task plan if active
/audit        → show last 10 audit log entries
/replay <session> → reconstruct a past session's tool chain
/metrics      → show session stats (calls, tokens, latency)
/thirsty <file> → run a .thirsty file
/tarl <file>   → evaluate a .tarl policy file
/lang <name>   → show info on a supported language
/langs        → list all languages in the knowledge base
/delegates    → list running sub-agent delegates
/pipeline <file> → execute a delegate pipeline definition file
/sandbox <backend> → switch sandbox backend (local/docker/ssh/singularity)
/skills       → list synthesized skills with use counts
/run-skill <name> → invoke a named skill
/provider <name> → switch model provider mid-session
/project      → show current project context profile
/gateway      → show platform gateway connection status

```

```
/clear          → clear screen
/exit           → save and exit
```

Implementation Order

StepFile(s)What

```
1git init T:\Dev\SovereignCreate repo
2.gitignore, pyproject.toml, CHANGELOG.mdRepo scaffolding + package definition
3install.ps1, install.shOne-command installers
4config.json, system_prompt.mdIdentity + configuration
5sovereign/config.pyConfig loader
6sovereign/security.pyPath validator + command filter
7sovereign/extracted/audit_log.pyTamperproof audit trail (extract + adapt)
6agent/extracted/event_spine.pyEvent bus (extract + adapt)
7agent/extracted/capability_token.pyToken system (extract + adapt)
8agent/extracted/intent_detection.pyIntent classifier (extract + adapt)
9agent/extracted/continuous_learning.pyFact absorber (extract + adapt)
10agent/extracted/ remaining 8 modulesAll other extracts
11agent/thirsty/Copy + adapt all 5 UTF tier packages
12agent/tools/filesystem.pyFile tools
13agent/tools/shell.pyExecution tools
14agent/tools/web.pyWeb tools
15agent/tools/code.pyCode/git tools
16agent/tools/system.pySystem tools
17agent/tools/data.pyData parsing + analysis
18agent/tools/thirsty.py8 Thirsty-Lang UTF tools
19agent/tools/language_db.py5 language database tools
20agent/sandbox/ all filesAll 5 sandbox backends + seccomp profile
21agent/delegates/rpc.pyJSON-RPC named pipe / TCP layer
22agent/delegates/templates/ all10 delegate templates
23agent/delegates/manager.py + pipeline.pyDelegateManager + Pipeline DSL
24agent/tools/delegates.py6 delegate management tools
25agent/tools/sandbox.pySandbox execution tool
26agent/tools/introspection.pyAudit, replay, metrics tools
27agent/projects/ all filesPer-project context detection + profile
28agent/providers/ all filesMulti-model provider router + 9 connectors
29agent/skills/ all filesPattern detector, synthesizer, tester, registry
30agent/gateway/ all filesGateway router + 6 platform connectors
31agent/tools/projects.py4 project context tools
32agent/tools/skills.py4 skill engine tools
33agent/tools/gateway.py2 gateway tools
34agent/tools/__init__.pyRegistry + dispatch (75+ tools registered)
35agent/memory/ all filesMemory layers + FAISS vector search
36agent/context.pyContext window management
37agent/router.pyModel router (intent + provider selection)
38agent/planner.pyTask planner
39agent/ui/terminal.pyRich TUI + Thirsty-Lang syntax highlighting
40agent/core.pyMain agentic loop
41sovereign/setup_wizard.pyInteractive setup wizard
42sovereign/updater.pysovereign update logic
43main.pyCLI entry point + all subcommands
44plugins/example_plugin.pyPlugin template
45scripts/populate_language_db.pyLanguage doc population script
46scripts/index_language_docs.pyLanguage FAISS indexer
47scripts/validate_language_db.pyDB completeness validator
48Seed languages/ with initial contentPython, JS, Rust, Go, SQL stubs first
49Run python scripts/index_language_docs.pyBuild language_docs FAISS index
50README.mdFull usage documentation
```

```

51.\install.ps1 end-to-end testRun full install wizard, verify sovereign command wor
52First git commitgit commit -m "feat: Sovereign v0.1.0 initial build"
Verification Sequence
.\install.ps1 - runs end-to-end, creates venv, installs deps, launches setup wizard
sovereign setup - wizard detects LM Studio, lets user pick models and paths, seeds
sovereign - agent starts, greets as "Sovereign" with model name + loaded tool count
sovereign update - pulls latest, re-indexes, prints changelog
"List files in T:\Dev\Sovereign" → list_directory fires, rich table output
"Write a Thirsty-Lang hello world to test.thirsty then run it" → write_file + run_thi
"Parse this TARL policy and tell me what it does" → eval_tarl_policy fires, explanati
"Validate this Shadow Thirst mutation" → validate_shadow_mutation, 6 analyzers run, \
"Write a Fibonacci script to test/fib.py then run it" → write_file + run_python chain
"What is my git status in T:\Dev\Sovereign" → git_status fires
"Remember that I prefer PowerShell over Bash" → save_memory + FAISS index updated
Exit, restart, ask "What shell do I prefer?" → semantic search recalls preference
"How does Rust handle memory ownership?" → query_language_docs(rust, memory ownerst
"Show me a Go concurrency example" → list_language_examples(go, concurrency) return
"Write me a Haskell function that filters a list" → agent uses haskell/syntax.md + ha
"Spawn a SecurityAuditor delegate on T:\Dev\Sovereign and report findings" → spawn_
"Run a pipeline: TestRunner then DocCompiler on T:\Dev\Sovereign" → run_pipeline ch
"Run this Python snippet in a Docker sandbox" → run_in_sandbox(docker, code=...) use
python main.py --project T:\Dev\Sovereign → project context loads, conventions.md i
Run same task 3 times → PatternDetector identifies it, SkillSynthesizer generates a
/run-skill <name> → synthesized skill executes successfully
python main.py --provider openrouter → agent connects to OpenRouter, lists availabl
Enable Telegram gateway in config → python main.py --gateway telegram → message sent
--resume <id> → session history reloads, conversation continues
/replay → deterministic replay reconstructs the tool call chain
/metrics → shows tool call counts, token estimates, latency averages, active provide

```